

PROJECT: Secure File Sharing with S3 and CloudFront

- **Services:** S3, CloudFront, IAM, Lambda@Edge
- **Description:** Implement a secure file-sharing service where users can upload files to S3, and CloudFront serves them with signed URLs.
- **Skills:** CDN setup, secure file access, IAM policies.

Definition

This project implements a **secure file-sharing system** using **Amazon S3** and **Amazon CloudFront**, where users can upload files to S3, and access them securely through **signed URLs** distributed via CloudFront.

Scope of the Project: The scope of this project is to design and implement a secure file delivery system using Amazon S3 and CloudFront. It ensures that files stored in S3 remain private and are accessible only through time-limited signed URLs.

The project includes configuring Origin Access Control, key groups, and cryptographic signing for secure access validation.

Each signed URL includes an expiration time, ensuring temporary and controlled access to protected resources.

Methodologies:

System Design Methodology:

Design the architecture using managed AWS services:

- Storage → Amazon S3
- CDN(Content Delivery Network) → Amazon CloudFront
- Access Control → AWS Identity and Access Management.

Techniques Used:

- Private S3 bucket (no public access)
- IAM roles & policies for controlled access
- CloudFront Origin Access Control (OAC/OAI)
- Signed URLs for temporary access

Services Used in this Project:

1. Amazon S3 (Simple Storage Service)

- **Definition:** Amazon S3 is a scalable object storage service used to store and retrieve data securely over the internet.
- **Purpose:** In this project, S3 is used to securely store private files that need controlled access.
- **Role in this Project:** S3 acts as the private origin storage layer that CloudFront accesses to deliver files through signed URLs.

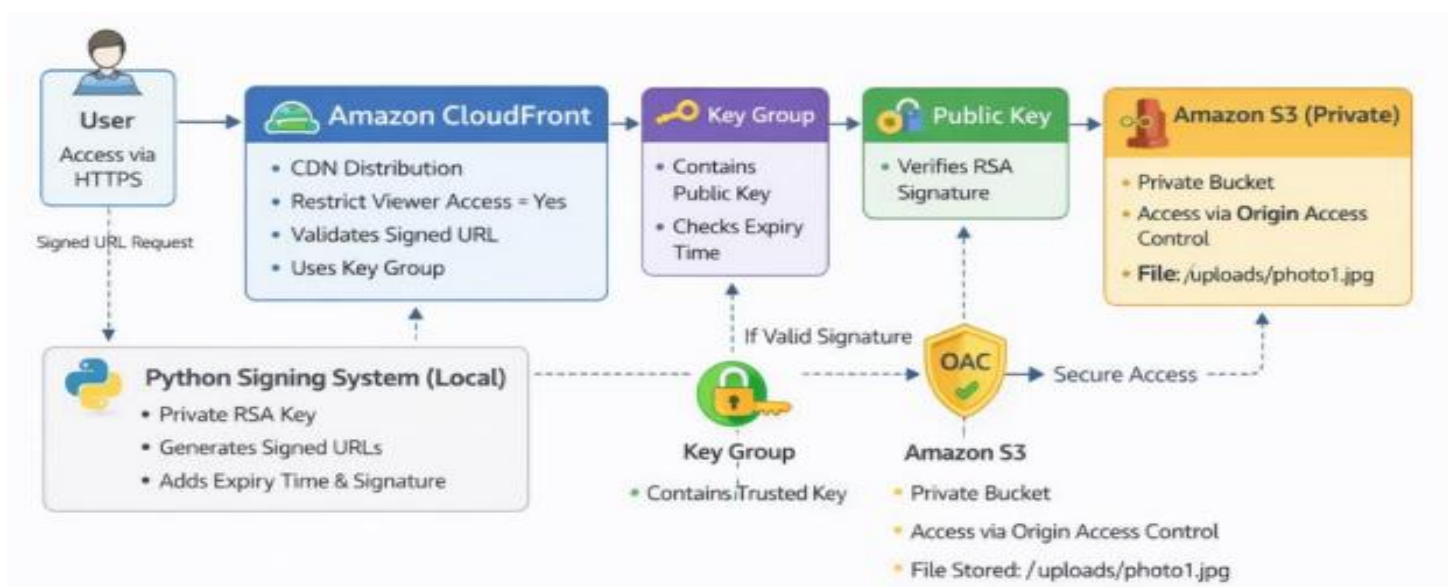
2. Amazon CloudFront:

- **Definition:** Amazon CloudFront is a Content Delivery Network (CDN) service provided by AWS that distributes content (like images, videos, APIs, and files) through a network of global edge locations to deliver data to users with low latency and high speed.
- **Purpose:** Used to serve files from nearest edge location, Users can access content from anywhere in the world, Supports **HTTPS**, signed **URLs**, and access restrictions, Caches content to minimize direct requests
- **Role in this Project:** CloudFront acts as the secure distribution layer that validates signed URLs and serves content from the private S3 bucket.

3. Amazon IAM (Identity and Access Management)

- **Definition:** AWS IAM is a service used to securely manage users, roles, and permissions in AWS.
- **Purpose:** In this project, IAM is used to control and grant secure permissions for accessing and configuring S3 and CloudFront resources.
- **Role in this Project:** IAM ensures that only authorized users and services can manage the S3 bucket, CloudFront distribution, and related security configurations.

Architecture Diagram:



Phase 1: Create S3 Bucket :

- General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1/1)

Info

Refresh

Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

< 1 > ⚙

Name	AWS Region	Creation date
secure-file-sharing-12345	Asia Pacific (Mumbai) ap-south-1	March 31, 2026, 08:38:08 (UTC+05:30)

- Created folder: /uploads
- Uploaded file: photo1.jpg

secure-file-sharing-12345 Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (1/1)

↻

Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▼

Create folder

⬆ Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐ Show versions

< 1 >

⚙

☒	Name	Type	Last modified	Size	Storage class
☒	uploads/	Folder	-	-	-

 Copy S3 URI

Objects

Properties

Objects (1/1)

↻

📄 Copy S3 URI

📄 Copy URL

⬇️ Download

🔗 Open

🗑️ Delete

⋮ Actions

📁 Create folder

⬆️ Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍

Show versions

< 1 >

⚙️

☒	Name	Type	Last modified	Size	Storage class
☒	 logo.jpg	jpg	March 31, 2026, 11:24:58 (UTC+05:30)	5.2 KB	Standard

Phase 3: Create CloudFront Distribution

- Go to CloudFront
- Click Create Distribution
- Origin: Select your S3 bucket
- Enable: Restrict viewer access = Yes
- Create Origin Access Control (OAC)
- Attach OAC to the origin • Create distribution

Distributions (1) [Info](#)

[Refresh](#) [Enable](#) [Disable](#) [Delete](#) [Create distribution](#)

Filter type
All distributions

< 1 > [Settings](#)

☒	ID	Status	Descrip...	Type	Domai...	Alterna...	Origins	Pricing ...	Last modified
☒	E2PRD9DEX4ITRS	Enabled	CloudFront...	Standard	d28j6...	-	secure-file-shar	Pay-as-yo...	March 31, 2026, 11:58 AM GMT+5:30

s3-file-delivery-distribution [Standard](#)

[View metrics](#)

Details

Distribution domain name

[d28j6k0pfvi87a.cloudfront.net](#)

Billing

[Billing](#)

ARN

[arn:aws:cloudfront::074139044318:distribution/E2PRD9DEX4ITRS](#)

Last modified

March 31, 2026 at 6:28:59 AM UTC

Phase 4: Create Public & Private Key Pair

- Go to CloudFront → Public Keys
- Create Public Key
- Upload RSA public key
- Save Public Key ID

Public keys

[Edit](#) [Delete](#) [Create public key](#)

☐	ID	Name	Type	Description
☐	K2JUW207K3GXHK	file-sharing-public-key	RSA 2048	-

Phase 5: Create Key Group

- Go to CloudFront → Key Groups
- Create Key Group
- Attach the Public Key
- Save Key Group

Key groups

[Edit](#) [Delete](#) [Create key group](#)

☒	ID	Name	Description	Last modified
☒	aacab0bb-6582-4123-9aff-2663609fbb51	file-sharing-group-key	-	March 31, 2026 at 6:10:22 AM UTC

Phase 6: Attach Key Group to CloudFront

- Go to CloudFront → Distribution
- Edit Behavior
- Under Trusted key groups
- Attach your Key Group
- Save changes
- Now CloudFront will:

- Validate Signed URL
- Check expiry
- Verify RSA signature

Restrict viewer access

If you restrict viewer access, viewers must use CloudFront signed URLs or signed cookies to access your content.

- ☐ No
☒ Yes

Trusted authorization type

- ☒ Trusted key groups (recommended)
☐ Trusted signer

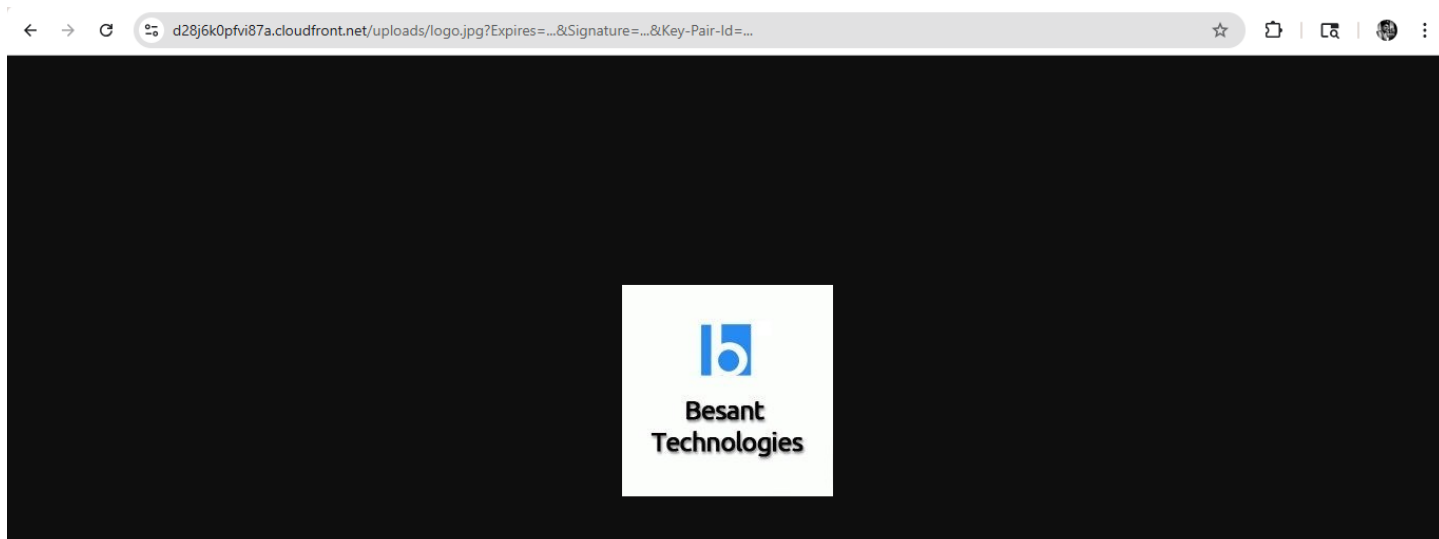
Add key groups

Choose the key groups

file-sharing-group-key (aacab0bb-6582-4123-9aff-2663609fbb51) X

[Create key group](#)

Final output:



Troubleshooting:

1. openssl not recognized in PowerShell
2. **Issue:** In Windows PowerShell, openssl version failed (openssl not installed / not in PATH).
3. **Solution:** I used google colab for download public key
 - from google.colab import files
 - files.download('public_key.pem')

Conclusion:

The Secure File Sharing with S3 and CloudFront project successfully demonstrates how to build a scalable, high-performance, and secure file delivery system using AWS services like Amazon S3, Amazon CloudFront, AWS Identity and Access Management.